

객체지향 프로그래밍

Object Oriented Programming

실세계의 모든 것들을 객체화 시켜 프로그래밍하는 것

객체 지향 프로그래밍(OOP: object-oriented programming)은 우리가 살고 있는 실제 세계가 객체(object)들로 구성되어 있는 것과 비슷하게, 소프트웨어도 객체로 구성하는 방법

구조적 프로그래밍과 다르게 큰 문제를 작게 쪼개는 것이 아니라,
먼저 작은 문제들을 해결 할 수 있는 객체들을 만든 뒤,
이 객체들을 조합해서 큰 문제를 해결하는 상향식(Bottom-up) 해결 프로그래밍 기법

절차지향 VS 객체지향

```
=====
학번 : 1001
학년 : 1
이름 : 박철수
전공 : 전자공학
성별 : 남
=====
학번 : 1002
학년 : 3
이름 : 이민정
전공 : 생물학과
성별 : 여
=====
학번 : 1003
학년 : 4
이름 : 김은철
전공 : 경영학과
성별 : 남
```

절차지향 VS 객체지향

- 코딩 기법 - 변수 활용

```
1 # 학생1
2 student_num_1 = 1001
3 student_grade_1 = 1
4 student_name_1 = '박철수'
5 student_major_1 = '전자공학'
6 student_gender_1 = '남'
```

```
1 # 학생2
2 student_num_2 = 1002
3 student_grade_2 = 3
4 student_name_2 = '이민정'
5 student_major_2 = '생물학과'
6 student_gender_2 = '여'
```

```
1 # 학생3
2 student_num_3 = 1003
3 student_grade_3 = 4
4 student_name_3 = '김은철'
5 student_major_3 = '경영학과'
6 student_gender_3 = '남'
```

절차지향 VS 객체지향

```
1  # 학생의 정보를 출력한다.
2  def printStudent(num, grade, name, major, gender):
3      print('='*30)
4      print(f'학번 : {num}')
5      print(f'학년 : {grade}')
6      print(f'이름 : {name}')
7      print(f'전공 : {major}')
8      print(f'성별 : {gender}')
9
10 printStudent(student_num_1, student_grade_1, student_name_1, student_major_1, student_gender_1)
11 printStudent(student_num_2, student_grade_2, student_name_2, student_major_2, student_gender_2)
12 printStudent(student_num_3, student_grade_3, student_name_3, student_major_3, student_gender_3)
```

절차지향 VS 객체지향

- 코딩 기법 - 리스트 구조

```
1 student_num_list = [1001, 1002, 1003]
2 student_grade_list = [1, 3, 4]
3 student_names_list = ['박철수', '이민정', '김은철']
4 student_major_list = ['전자공학', '생물학과', '경영학과']
5 student_gender_list = ['남', '여', '남']
```

절차지향 VS 객체지향

```
1 def printStudentList(num, grade, name, major, gender):
2     for i in range(len(student_num_list)):
3         print('='*30)
4         print(f'학번 : {num[i]}')
5         print(f'학년 : {grade[i]}')
6         print(f'이름 : {name[i]}')
7         print(f'전공 : {major[i]}')
8         print(f'성별 : {gender[i]}')
9
10 printStudentList(student_num_list, student_grade_list,
11                  student_names_list, student_major_list,
12                  student_gender_list)
```

절차지향 VS 객체지향

- 코딩 기법 - 리스트안에 딕셔너리 구조

```
1 students_dict_list = [  
2     {'num': 1001, 'grade': 1, 'name': '박철수', 'major': '전자공학', 'gender': '남'},  
3     {'num': 1002, 'grade': 3, 'name': '이민정', 'major': '생물학과', 'gender': '여'},  
4     {'num': 1003, 'grade': 4, 'name': '김은철', 'major': '경영학과', 'gender': '남'}  
5 ]
```

절차지향 VS 객체지향

```
1 def printStudentDict(dict_list):
2     for student in students_dict_list:
3         print('='*30)
4         print(f"학번 : {student['num']}")
5         print(f"학년 : {student['grade']}")
6         print(f"이름 : {student['name']}")
7         print(f"전공 : {student['major']}")
8         print(f"성별 : {student['gender']}")
9
10 printStudentDict(students_dict_list)
```


절차지향 VS 객체지향

- 코딩 기법 - 클래스 구조

```
1 class Student():
2     def __init__(self, num, grade, name, major, gender):
3         self.num = num
4         self.grade = grade
5         self.name = name
6         self.major = major
7         self.gender = gender
8
9     def student_print(self):
10        print('='*30)
11        print(f"학번 : {self.num}")
12        print(f"학년 : {self.grade}")
13        print(f"이름 : {self.name}")
14        print(f"전공 : {self.major}")
15        print(f"성별 : {self.gender}")
```

절차지향 VS 객체지향

```
1 student1 = Student(1001, 1, '박철수', '전자공학', '남')
2 student2 = Student(1002, 3, '이민정', '생물공학', '여')
3 student3 = Student(1003, 4, '김은철', '경영학과', '남')
4
5 student1.student_print()
6 student2.student_print()
7 student3.student_print()
```

클래스

- 클래스(Class)란?

똑같은 무엇인가를 계속해서 만들어낼 수 있는 설계 도면
다수의 변수와 다수의 함수 집합
고유한 특성안에서 변경은 가능

- 클래스(Class)의 장점

1. 코드의 재사용
2. 구조화
3. 업데이트 용이
4. 코딩의 단순화

과자틀 → 클래스 (class)

과자틀에 의해서 만들어진 과자들
→ 객체 (object)

클래스

- 클래스 정의

클래스이름은 첫 글자는 대문자로 지정

class 클래스이름:
 명령문

```
class Square:  
    pass
```

```
print(type(Square))  
<class 'type'>
```

```
<class '__main__.Car'> <class 'type'>
```

객체

- 인스턴스(Instance)란?

"구체적인 사례로 만들다"

- 객체(Object)란?

- 클래스를 인스턴스화한 것
- 클래스에 의해서 만들어진 피조물
- 인간 클래스의 인스턴스화한 것(=객체)은 홍길동이다

객체

- 객체(또는 인스턴스) 생성

객체명 = 클래스()

객체명은 첫글자는 소문자로 지정

자동차 클래스 정의

```
class Car:
```

```
    pass
```

```
print(Car, type(Car))
```

자동차 클래스에서 객체 생성

```
car1 = Car()
```

```
car2 = Car()
```

```
print(car1, type(car1))
```

```
print(car2, type(car2))
```

```
<class '__main__.Car'> <class 'type'>  
<__main__.Car object at 0x0000016ECDEFA1B0> <class '__main__.Car'>  
<__main__.Car object at 0x0000016ECDEFA210> <class '__main__.Car'>
```

속성과 기능

객체(Object) = 속성(Attribute) + 기능(Method)

- 속성은 사물의 특징

예) 자동차의 속성 : 바디의 색, 바퀴의 크기, 엔진의 배기량

- 기능은 어떤 것의 특징적인 동작

예) 자동차의 기능 : 전진, 후진, 좌회전, 우회전

- 속성과 기능을 들어 자동차를 묘사하면?

"18인치의 바퀴를 가진 2,000cc의 빨간 차는 전진, 후진, 좌회전, 우회전의 기능이 있다."

속성 정의

생성자함수를 정의하지 않고 객체 생성 후 값을 할당하는 방식
객체.속성명 = 속성에 넣을 값

```
class User:  
    pass
```

```
user1 = User()  
user2 = User()
```

```
user1.name = "둘리"  
user1.email = "user1@test.com"  
user1.address = "서울"
```

```
user2.name = "마이콜"  
user2.email = "user2@test.com"  
user2.address = "구리"
```

```
print(f' user1의 정보 : 이름 = > {user1.name}, 이메일 => {user1.email}, 지역 => {user1.address}')  
print(f' user2의 정보 : 이름 = > {user2.name}, 이메일 => {user2.email}, 지역 => {user2.address}')
```

```
user1의 정보 : 이름 = > 둘리, 이메일 => user1@test.com, 지역 => 서울  
user2의 정보 : 이름 = > 마이콜, 이메일 => user2@test.com, 지역 => 구리
```


생성자 메소드

- 객체가 생성될 때 여러가지 초기화작업을 할 때 유용하게 사용할 수 있다.
- 메소드명은 '`__init__`' 이다.
- 클래스가 인스턴스화(객체화) 될 때 호출된다.
- 항상 첫번째 매개변수 인자는 `self` 이다

```
class 클래스명:  
    def __init__(self, 인자1, 인자2...):  
        self.속성 = 인자값
```

```
객체명 = 클래스(값1, 값2....)
```

생성자 메소드

```
class Square:
```

```
    def __init__(self,height,width):
```

```
        self.height = height
```

```
        self.width = width
```

```
square = Square(50,100)
```

```
print('square1의 높이는 {} 입니다'.format(square.height))
```

```
print('square1의 가로 크기는 {} 입니다'.format(square.width))
```

```
print('square1의 넓이는 {} 입니다'.format(square.width*square.height))
```

```
square1의 높이는 50 입니다
```

```
square1의 가로 크기는 100 입니다
```

```
square1의 넓이는 5000 입니다
```

메소드(Method)

- 클래스에서 정의된 코드블록의 묶음.
- 함수와 비슷하다.

```
class 클래스이름:  
    def 메소드이름(self):  
        코드블록  
        return value
```

- 인스턴스화

```
객체이름 = 클래스이름(값 ... )  
객체이름.메소드이름(값 ...)
```

메소드(Method)

사각형 클래스 정의

속성 : 가로, 세로, 색상, 제조자(영희, 철수)

메서드 : 사각형의 넓이 출력 , 사각형의 속성 출력

메소드(Method)

```
class Square:
    # 생성자 메서드 정의
    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        self.color = color
        self.maker = ('영희', '철수')

    # 사각형의 넓이 출력 메서드
    def area(self):
        print(f' 사각형의 넓이는? {self.width * self.height}')

    # 사각형의 속성 출력 메서드
    def print_info(self):
        print(f' 가로 : {self.width} 세로 : {self.height} 색상 : {self.color}')
        print(f' 만든사람 : {self.maker}')
```

메소드(Method)

```
square1 = Square(10, 10, 'red')
```

```
print('square1 속성 출력')
```

```
print(f' 가로 : {square1.width} 세로 : {square1.height} 색상 : {square1.color}')
```

```
print(f' 만든사람 : {square1.maker}')
```

```
print(f' 만든사람 : {square1.maker[0]} 와 {square1.maker[1]}')
```

```
square1.area()
```

```
square1.print_info()
```

square1 속성 출력

가로 : 10 세로 : 10 색상 : red

만든사람 : ('영희', '철수')

만든사람 : 영희 와 철수

사각형의 넓이는? 100

가로 : 10 세로 : 10 색상 : red

만든사람 : ('영희', '철수')

메소드(Method)

```
square2 = Square(50, 30, 'blue')
```

```
print('square2 속성 출력')
print(f' 가로 : {square2.width} 세로 : {square2.height} 색상 : {square2.color}')
print(f' 만든사람 : {square2.maker}')
print(f' 만든사람 : {square2.maker[0]} 와 {square2.maker[1]}')
print()
square2.area()
square2.print_info()
```

```
square2 속성 출력
가로 : 50 세로 : 30 색상 : blue
만든사람 : ('영희', '철수')
만든사람 : 영희 와 철수
```

```
사각형의 넓이는? 1500
가로 : 50 세로 : 30 색상 : blue
만든사람 : ('영희', '철수')
```

퀴즈

계산기 Calculator 클래스 만들기

2개의 숫자를 속성으로 가진 계산기

4개의 메소드 (더하기 / 빼기 / 곱하기 / 나누기)

인스턴스화 시킨 후 다음과 같이 출력한다

```
첫번째 숫자 : 10
두번째 숫자 : -6
=====
더하기 : 4
빼기 : 16
곱하기 : -60
나누기 : -2
```


메소드에 별도의 인자가 있는 경우의 클래스

Bread 클래스

속성 => 빵이름, 가격, 칼로리, 재료, 브랜드

메서드 => 빵정보 출력, 주문한 빵 갯수에 따른 가격 출력

종류 => 모카빵

가격 => 4000

칼로리 => 660

재료 => ('밀가루', '설탕', '향신료', '버터')

브랜드 => 파리바게뜨

모카빵 를(을) 3 개 주문하셨습니다.

주문 가격은? 4000 X 3 = 12000

메소드에 별도의 인자가 있는 경우의 클래스

```
class Bread:
    def __init__(self, kind, price, kcal, src):
        self.kind = kind
        self.price = price
        self.kcal = kcal
        self.src = src
        self.brand = '파리바게뜨'

    def print_info(self):
        print(f'종류 => {self.kind}')
        print(f'가격 => {self.price}')
        print(f'칼로리 => {self.kcal}')
        print(f'재료 => {self.src}')
        print(f'브랜드 => {self.brand}')

    def order_info(self, n):
        print()
        print(f'{self.kind} 를(을) {n} 개 주문하셨습니다. ')
        print(f'주문 가격은? {self.price} X {n} = {self.price*n}')
```

```
bread1 = Bread('모카빵', 4000, 660, ('밀가루', '설탕', '향신료', '버터'))
bread2 = Bread('맘모스빵', 5500, 1200, ('밀가루', '설탕', '버터', '우유'))
print('='*50)
bread1.print_info()
bread1.order_info(3)
print('='*50)
bread2.print_info()
bread2.order_info(12)
```

```
=====
종류 => 모카빵
가격 => 4000
칼로리 => 660
재료 => ('밀가루', '설탕', '향신료', '버터')
브랜드 => 파리바게뜨
```

```
모카빵 를(을) 3 개 주문하셨습니다.
주문 가격은? 4000 X 3 = 12000
```

```
=====
종류 => 맘모스빵
가격 => 5500
칼로리 => 1200
재료 => ('밀가루', '설탕', '버터', '우유')
브랜드 => 파리바게뜨
```

```
맘모스빵 를(을) 12 개 주문하셨습니다.
주문 가격은? 5500 X 12 = 66000
```

퀴즈

Cat 클래스를 만들고 객체화 한 후 메서드를 호출하여라

속성 : kind, name, age, gender, animal_kind(고양이)

메서드 : 속성출력(print_info), 동작 : run, sleep(어디서), eat(무엇을)

객체 예시

```
cat1 = Cat('코캣', '딩치', 1, '남')
```

```
cat2 = Cat('러시안블루', '나비', 5, '여')
```

메서드 호출 예시

```
# cat1.print_info()
```

종류 = 러시안블루

이름 = 나비

나이 = 5

성별 = 여

퀴즈

메서드 호출 예시

cat1.run()

cat2.run()

cat1.sleep('캣타워')

cat2.sleep('택배 박스')

cat1.eat('사료')

cat2.eat('츄스')

고양이 덩치 가 달린다.

고양이 나비 가 달린다.

고양이 덩치가 캣타워에서 잔다.

고양이 나비가 택배 박스에서 잔다.

고양이 덩치가 사료을(를) 먹는다.

고양이 나비가 츄스을(를) 먹는다

퀴즈

메서드 호출 예시

```
cat1.action_print()
```

고양이 덩치 의 아침 일상

고양이 덩치가 물을(를) 먹는다.

고양이 덩치가 사료를(를) 먹는다.

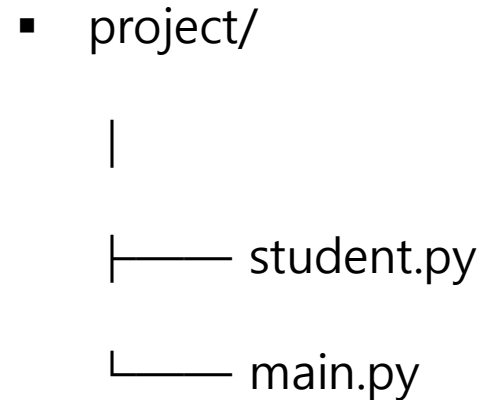
고양이 덩치 가 달린다.

고양이 덩치가 간식을(를) 먹는다.

고양이 덩치가 쇼파에서 잔다.

모듈 분리(import) 와 클래스 사용법

- 클래스의 모듈 분리
- import를 이용한 클래스 사용
- 객체 리스트와 반복문 활용



모듈 분리(import) 와 클래스 사용법

```
# student.py
```

```
class Student:
```

```
    def __init__(self, name, major, grade):
```

```
        self.name = name
```

```
        self.major = major
```

```
        self.grade = grade
```

```
    def show_info(self):
```

```
        print(f"이름 : {self.name}")
```

```
        print(f"학과 : {self.major}")
```

```
        print(f"학년 : {self.grade}학년")
```

```
    def study(self, subject):
```

```
        print(f"{self.name} 학생이 {subject}을(를) 공부합니다.")
```


모듈 분리(import) 와 클래스 사용법

```
# main.py
```

```
from student import Student
```

```
student1 = Student("홍길동", "컴퓨터공학과", 2)
```

```
student2 = Student("김철수", "AI학과", 1)
```

```
students = [student1, student2]
```

```
for student in students:
```

```
    student.show_info()
```

```
    student.study("파이썬")
```

```
    print("-" * 30)
```

모듈 분리(import) 와 클래스 사용법

이름 : 홍길동
학과 : 컴퓨터공학과
학년 : 2학년
홍길동 학생이 파이썬을(를) 공부합니다.

이름 : 김철수
학과 : AI학과
학년 : 1학년
김철수 학생이 파이썬을(를) 공부합니다.

클래스 변수

- 클래스 정의시 지정된 공용 변수
- 생성자 함수 위에 정의
- 동일한 주소값을 가지고 있다.

id(객체이름) : 주소 표시

- 인스턴스 생성 없이도 접근 가능

클래스명.클래스변수명

- 인스턴스 생성 후 접근도 가능

인스턴스명.클래스변수명

클래스 변수

```
class Test:
    # 클래스 변수 정의
    message = "오늘도 좋은 하루"

    def __init__(self, userName):
        self.userName = userName
        self.city = '부산'

    def messagePrint(self):
        print(f'{self.userName} 님!! {self.message}')

t1 = Test('김철수')
t1.messagePrint()
```

인스턴스화 한 후 속성 접근
print(t1.city)

id 값 확인
print(Test.message, id(Test.message))
print(t1.message, id(t1.message))

```
김철수 님!! 오늘도 좋은 하루
오늘도 좋은 하루 1714732828752
오늘도 좋은 하루 1714732828752
```

클래스 변수

```
class Car :  
    count = 0  
  
    def __init__(self, color, speed) :  
        self.speed = speed  
        self.color = color  
        Car.count += 1  
  
car1 = Car('black', 30)  
print(f"자동차 {car1.color} 의 현재 속도는 {car1.speed}km, 생산된 자동차는 총 {Car.count}  
대입니다.")  
  
car2 = Car('red', 60)  
car3 = Car('blue', 80)  
print(f"자동차 {car3.color} 의 현재 속도는 {car3.speed}km, 생산된 자동차는 총 {Car.count}  
대입니다.")
```

자동차 black 의 현재 속도는 30km, 생산된 자동차는 총 1대입니다.
자동차 blue 의 현재 속도는 80km, 생산된 자동차는 총 3대입니다.

퀴즈

도서 관리와 관련된 클래스 Library를 프로그래밍 하여라.

- 전체 도서 수와 도서 제목을 관리하는 클래스 변수 이용

현재 도서관에 있는 책의 개수: 3

도서관에 등록된 모든 책의 제목:

1. 파이썬 자료구조
2. 알고리즘 개론
3. 데이터 과학 입문

퀴즈

```
class Library:
    # 클래스 변수: 전체 도서 수와 도서 제목을 관리
    total_books = 0
    book_titles = [] # 도서 제목을 저장할 리스트

    def __init__(self, title, author):
        # 생성자 메서드 프로그래밍

    def show_total_books(self):
        # 현재 라이브러리의 전체 도서 총권수 출력 프로그래밍

    def show_all_titles(self):
        # 모든 도서의 제목 출력 프로그래밍
```

퀴즈

도서 추가

```
book1 = Library("파이썬 자료구조", "김철수")
```

```
book2 = Library("알고리즘 개론", "이영희")
```

```
book3 = Library("데이터 과학 입문", "박민준")
```

현재 도서 개수 출력

```
book1.show_total_books()
```

모든 도서 제목 출력

```
book1.show_all_titles()
```


매직 메소드 (Magic Methods)

- 매직 메소드(혹은 던더 메소드, Dunder Method)는 `__init__`, `__del__` 같은 특별한 역할을 하는 메소드
- `__이름__` 형태로 되어 있으며, 연산자 오버로딩, 객체 표현, 비교 연산 등을 구현할 때 사용된다

매직 메소드 (Magic Methods)

매직 메소드	설명
<code>__init__()</code>	생성자
<code>__del__()</code>	소멸자
<code>__str__()</code>	<code>print(obj)</code> 호출 시 문자열 반환
<code>__repr__()</code>	<code>repr(obj)</code> 호출 시 문자열 반환
<code>__add__()</code>	<code>+</code> 연산자 오버로딩
<code>__sub__()</code>	<code>-</code> 연산자 오버로딩
<code>__mul__()</code>	<code>*</code> 연산자 오버로딩

매직 메소드 (Magic Methods)

<code>__eq__()</code>	<code>==</code> 비교 연산 오버로딩
<code>__lt__()</code>	<code><</code> 비교 연산 오버로딩
<code>__gt__()</code>	<code>></code> 비교 연산 오버로딩
<code>__getitem__()</code>	<code>obj[key]</code> 호출 시 동작
<code>__setitem__()</code>	<code>obj[key] = value</code> 호출 시 동작
<code>__delitem__()</code>	<code>del obj[key]</code> 호출 시 동작

매직 메소드 (Magic Methods)

생성자 (`__init__`): 객체가 생성될 때 자동으로 실행되는 메서드

소멸자 (`__del__`): 객체가 삭제될 때 자동으로 실행되는 메서드

```
class MyClass:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} 객체가 생성되었습니다.")
    def __del__(self):
        print(f"{self.name} 객체가 소멸되었습니다.")
```

매직 메소드 (Magic Methods)

객체 생성

```
obj1 = MyClass("객체1")
```

```
obj2 = MyClass("객체2")
```

객체 삭제

```
del obj1
```

```
print("프로그램 종료")
```

객체1 객체가 생성되었습니다.
객체2 객체가 생성되었습니다.
객체1 객체가 소멸되었습니다.
프로그램 종료

상속(Inheritance)

- 물려받다
- 어떤 클래스를 만들 때 다른 클래스의 기능을 물려받을 수 있게 만드는 것
- 객체지향의 가장 큰 장점은 코드의 재사용(재활용)
- 파이썬 클래스는 자바와는 다르게 다중 클래스를 지원한다.
- 부모클래스=슈퍼클래스=조상클래스
- 자식클래스=서브클래스=파생클래스

class 클래스명(상속할 클래스명):
 메소드 또는 속성

상속(Inheritance)

부모 클래스

```
class Car():
```

```
    def __init__(self, door):
```

```
        self.speed = 0
```

```
        self.door = door
```

```
    def upSpeed(self, speed):
```

```
        self.speed += speed
```

```
        print()
```

```
        print(f"현재 속도 : {self.speed}")
```

자식 클래스

```
class Sedan(Car):
```

```
    def __init__(self, speed, door):
```

```
        # 부모 클래스의 생성자 메서드 호출
```

```
        Car.__init__(self, door)
```

```
        self.speed = speed
```

자식 클래스에서 추가한 메서드

```
    def downSpeed(self, speed):
```

```
        self.speed -= speed
```

```
        print(f"현재속도(자식 클래스) : {self.speed}")
```

상속(Inheritance)

```
car = Car(4)
print(f"현재 속도(부모 클래스) : {car.speed}")
print(f"문의 갯수(부모 클래스) : {car.door}")
car.upSpeed(50)
```

현재 속도(부모 클래스) : 0
문의 갯수(부모 클래스) : 4

현재 속도 : 50

```
sedan = Sedan(100, 2)
print(f"현재 속도(자식 클래스) : {sedan.speed}")
print(f"문의 갯수(자식 클래스) : {sedan.door}")
sedan.upSpeed(120)
sedan.downSpeed(30)
```

현재 속도(자식 클래스) : 100
문의 갯수(자식 클래스) : 2

현재 속도 : 220
현재속도(자식 클래스) : 190

다중 클래스 상속

- 부모클래스를 상속해서 자식 클래스 만들기

class 자식클래스명(부모클래스명1, 부모클래스명2....):
 명령문

2개의 부모 클래스 생성후 자식 클래스 상속하기

```
class Papa:
    firstName = '김'
    def info1(self):
        print('아파트, 차')
```

```
class Mama:
    familyName = '이'
    def info2(self):
        print('오피스텔')
```

자식 클래스 정의

```
class Child(Papa, Mama):
    def __init__(self, myname):
        Papa.__init__(self)
        Mama.__init__(self)
        self.myname = myname
```

```
    def info3(self):
        print('골프 회원권')
```

다중 클래스 상속

```
# 부모 클래스 상속 확인
```

```
child = Child('철수')
```

```
child.info1()
```

```
child.info2()
```

```
child.info3()
```

```
print(f'자식 클래스의 이름은 ... {child.firstName} {child.familyName} {child.myname} 입니다.' )
```

아파트, 차

오피스텔

골프 회원권

자식 클래스의 이름은 ... 김 이 철수 입니다.

퀴즈

Person과 Employee 클래스를 상속받는 Manager 클래스를 정의하여라.

- Person 클래스는 name과 age 속성을 가지고 있고, introduce 메서드를 정의한다
- Employee 클래스는 position과 salary 속성을 가지고 있고, work 메서드를 정의한다.
- Manager 클래스는 Person과 Employee를 상속받고, manage라는 새로운 메서드를 추가한다

안녕하세요, 제 이름은 홍길동이고 35살입니다.

매니저로 일하고 있습니다.

홍길동는 매니저로서 팀을 관리하고 있습니다.

퀴즈

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"안녕하세요, 제 이름은 {self.name}이고 {self.age}살입니다.")

class Employee:
    def __init__(self, position, salary):
        self.position = position
        self.salary = salary

    def work(self):
        print(f"{self.position}로 일하고 있습니다.")
```

퀴즈

```
class Manager(Person, Employee):
```

```
    # 프로그래밍
```

```
# 객체 생성
```

```
manager = Manager("홍길동", 35, "매니저", 5000)
```

```
# 메서드 호출 프로그래밍
```

메소드 오버라이딩

- 부모 클래스의 메서드를 자식 클래스에서 재정의하는 것
- `super()` 메서드

자식 클래스에서 메서드 오버라이딩을 할 때,
조상 클래스의 메서드나 속성을 사용해야 하는 경우가 있다.
이럴때는 `super()` 메서드를 사용하면 된다.

메소드 오버라이딩

부모 클래스 : Tiger, Lion
자식 클래스 : Liger

```
class Tiger:
    kind = '호랑이'
    def jump(self):
        print(f'{Tiger.kind} 처럼 점프하기')
    def cry(self):
        print(f'{self.kind} : 어흥 ~')

class Lion:
    kind = '사자'
    def bite(self):
        print(f'{self.kind} 처럼 한입에 꿀꺽하기')
    def cry(self):
        print(f'{self.kind} : 으르렁 ~')
```

메소드 오버라이딩

```
class Liger(Tiger, Lion):
    kind = '라이거'

    def __init__(self, name):
        Tiger.__init__(self)
        Lion.__init__(self)
        self.name = name

    def play(self):
        print(f'{self.kind} 처럼 사육사와 놀기')

    # 메서드 오버라이딩
    def jump(self):
        print(f'{self.kind} 처럼 달리면서 점프하기~')

    def jump_papa(self):
        super().jump()
```


메소드 오버라이딩

```
liger = Liger('덩치')  
print(f"자식 클래스 이름 : {liger.name} Wt 종류 : {liger.kind}")  
liger.cry()  
liger.play()  
liger.bite()  
liger.jump()  
liger.jump_papa()
```

자식 클래스 이름 : 덩치	종류 : 라이거
라이거 : 어흥 ~	
라이거 처럼 사육사와 놀기	
라이거 처럼 한입에 꿀꺽하기	
라이거 처럼 달리면서 점프하기~	
호랑이 처럼 점프하기	

퀴즈

부모 클래스 Shape를 상속받아 여러 도형을 생성하고, 각 도형의 넓이를 계산하도록 구현하여라.

Shape 클래스

- : 속성 shape_name 도형의 이름
- : 메서드 show_info() 도형의 이름 출력
- area() 도형의 넓이를 반환

Rectangle 클래스

- : Shape 클래스를 상속받는다.
- : 속성 - 도형 이름, 가로(width), 세로(height)
- : area() 메서드를 오버라이딩하여 사각형의 넓이를 반환

퀴즈

Circle 클래스

- : Shape 클래스를 상속받는다.
- : 속성 - 도형 이름, 반지름(radius)
- : area() 메서드를 오버라이딩하여 원의 넓이를 반환

Triangle 클래스

- : Shape 클래스를 상속받는다.
- : 속성 - 도형 이름, 밑변(base), 높이(height)
- : area() 메서드를 오버라이딩하여 삼각형의 넓이를 반환

- Shape 클래스를 반드시 부모 클래스로 사용한다.
- Rectangle, Circle, Triangle 클래스는 Shape 클래스를 상속받는다.
- 각 자식 클래스는 area() 메서드를 반드시 오버라이딩한다.
- 부모 클래스의 show_info() 메서드는 모든 자식 객체에서 그대로 사용할 수 있어야 한다.
- 도형 객체는 리스트에 저장한 후 반복문으로 처리한다.

퀴즈

```
import math

class Shape:
    def __init__(self, shape_name):
        self.shape_name = shape_name

    def show_info(self):
        print(f"도형 : {self.shape_name}")

    def area(self):
        pass
```

퀴즈

실행

```
shapes = [ Rectangle(5, 4), Circle(3) , Triangle(5, 10) ]
```

for shape in shapes:

```
    shape.show_info()
```

```
    print(f"넓이 : {shape.area():.2f}")
```

```
    print("-" * 20)
```

도형 : 사각형

넓이 : 20.00

도형 : 원

넓이 : 28.27

도형 : 삼각형

넓이 : 25.00
